

(long long) 对整数点积性能的影响

对照实验：原版使用 (long long) 强制类型转换，对照版去掉类型转换。
两个版本在同一台机器上连续运行，编译器均为 gcc -O3 (Apple Silicon ARM64)。

原版：

```
long long sum = 0;
sum += (long long)a[i] * b[i];
```

对照版（无类型转换）：

```
int sum = 0;
sum += a[i] * b[i];
```

完整数据 (C -O3)

int

n	有类型转换 (ms)	无类型转换 (ms)	加速比
128	0.000006	0.000005	1.1×
1,000	0.000043	0.000041	1.0×
10,000	0.000466	0.000418	1.1×
100,000	0.00569	0.00571	1.0×
1,000,000	0.0668	0.0658	1.0×
10,000,000	0.963	0.974	1.0×
100,000,000	9.746	10.485	0.9×

short

n	有类型转换 (ms)	无类型转换 (ms)	加速比
128	0.000021	0.000003	7.0×
1,000	0.000170	0.000022	7.7×
10,000	0.00172	0.000230	7.5×
100,000	0.0174	0.00294	5.9×
1,000,000	0.195	0.0322	6.1×
10,000,000	2.053	0.754	2.7×
100,000,000	17.365	4.908	3.5×

char

n	有类型转换 (ms)	无类型转换 (ms)	加速比
128	0.000020	0.000004	4.7×
1,000	0.000173	0.000030	5.7×
10,000	0.00172	0.000295	5.8×
100,000	0.0181	0.00289	6.3×
1,000,000	0.178	0.0283	6.3×
10,000,000	1.770	0.285	6.2×
100,000,000	26.137	11.507	2.3×

分析 1：int 几乎无差异

gcc -O3 -S 导出汇编，对比 DotInt 主循环：

```
;; 有类型转换 → smlal.2d (32→64 widening, 每寄存器 2 lane)
smlal.2d  v0, v20, v16          ; 8 条 smlal → 每轮 16 个元素

;; 无类型转换 → mla.4s (32→32 同宽, 每寄存器 4 lane)
mla.4s    v0, v16, v4           ; 4 条 mla → 每轮 16 个元素
```

两者都处理 16 元素/轮 (long long 版用 8 条 smlal.2d , int 版用 4 条 mla.4s)。虽然 int 版指令数更少, 但 CPU 流水线可以同时执行多条 smlal 指令 (它们操作不同的寄存器, 互不依赖), 掩盖了指令数差异。

分析 2: short/char 差距大

有类型转换时, DotShort 汇编:

```
;; 编译器放弃向量化, 退回标量
ldrsh    x17, [x15, #-4]        ; 逐个加载 1 个 short, 符号扩展到 64 位
smaddl   x11, w5, w17, x11      ; 标量乘加
subs     x16, x16, #4           ; 每轮 4 个元素
```

无类型转换时, 同一个 DotShort:

```
;; 成功向量化
ldp      q16, q17, [x11, #-32]  ; 批量加载 16 个 short
smlal.4s  v0, v20, v16          ; SIMD 16→32 widening 乘加
subs     x12, x12, #32          ; 每轮 32 个元素
```

4 个/轮 vs 32 个/轮 = 8 倍吞吐量差距。

原因: 根据 [ARM Architecture Reference Manual](#), SMLAL (Signed Multiply-Accumulate Long) 指令的输出宽度固定为输入宽度的两倍, 不存在跨多级 widening 的变体:

跳转	指令	支持
8→16 (2×)	smlal.8h	✓
16→32 (2×)	smlal.4s	✓
32→64 (2×)	smlal.2d	✓

跳转	指令	支持
16→64 (4×)	—	×
8→64 (8×)	—	×

(long long) 类型转换使 short 需要 16→64 跳转 (翻四倍)，没有对应指令，编译器退回标量。去掉后变成 16→32 (翻倍)，匹配 smlal.4s，成功向量化。

小 n 时加速比最大 (~7×，接近 8 倍吞吐差距)，大 n 时下降到 2-4×，因为内存带宽逐渐成为瓶颈。

结论

原版报告中 int 比 short/char 快，真正原因不是 int 运算天然快，而是 (long long) 类型转换阻碍了 short/char 的 SIMD 向量化。